

第 11 讲

近似算法



主要内容

- 11.1. 近似算法基本概念
- 11.2 多项式近似方案
- 11.3 近似算法的典型应用例
 - 顶点覆盖
 - 旅行商问题
 - 01 背包问题
- 11.4 近似化策略总结



11.1 近似算法基本概念

- 近似算法通常与 NP 问题相关，由于目前不可能采用有效的多项式时间精确地解决 NP 问题，所以采用多项式时间求一个次优解。
- 理想情况下，近似值最优可达到一个小的常数因子（例如在最优解的 5% 以内）。近似算法越来越多地用于已知精确多项式时间算法但由于输入大小而过于昂贵的问题。

11.1 近似算法基本概念

- 时间复杂性必须是多项式阶的。这是近似算法的基本目标。
- 近似算法的重要目标——解的近似程度。
- 近似比：最优化问题的最优值为 c^* ，求解该问题的一个近似算法求得的近似最优值为 c ，

$$\begin{array}{ccc} \text{最小化问题} & \leftarrow \eta = \max \left\{ \frac{c}{c^*}, \frac{c^*}{c} \right\} \rightarrow & \text{最大化问题, } c^* \geq c \\ c \geq c^* & & \eta \geq 1; \text{ 且 } \eta \text{ 越大, 近似解越差!} \end{array}$$

- 与问题规模 n 的关系，

$$\max \left\{ \frac{c}{c^*}, \frac{c^*}{c} \right\} \leq \rho(n)$$

11.1. 近似算法基本概念

- 相对误差 λ

$$\lambda = \left| \frac{c - c^*}{c^*} \right|$$

- 若问题的输入规模为 n ，存在一个函数 $\epsilon(n)$ ，使得：

$$\left| \frac{c - c^*}{c^*} \right| \leq \epsilon(n)$$

$\epsilon(n)$ 称为近似算法的相对误差界。且有 $\epsilon(n) \leq \rho(n) - 1$

11.2 多项式近似方案

- 最优化问题的近似方案是一个近似算法
 - 给定一个输入实例，有一个值 $\epsilon > 0$ ，使得对于任何固定的 ϵ ，该方案是个 $(1 + \epsilon)$ 近似算法。
 - 对一个近似方案来说，如果对于任何固定的 $\epsilon > 0$ ，该方案都以其输入实例的规模 n 的多项式时间允许，则此方案为多项式时间近似方案。

11.2 完全多项式近似方案

- 例如运行时间是 $O(n^{\frac{2}{\epsilon}})$ ， ϵ 减少，多项式时间近似方案的运行时间会迅速增长。
- 运行时间既是 $\frac{1}{\epsilon}$ 的多项式，又是 n 的多项式。
- 如果一个近似方案，其运行时间既是 $\frac{1}{\epsilon}$ 的多项式，又是输入实例规模 n 的多项式，则称其为完全多项式时间的近似方案。



11.3 近似算法的典型应用——顶点覆盖问题

- 给定一个无向图 $G=(V,E)$ ，图 G 的一个顶点覆盖是节点集合 V 的一个子集 V_{sub} ，对于任意一条边 (u,v) ，满足下面的三个条件之一
 - $u \in V_{sub}$
 - $v \in V_{sub}$
 - $u \in V_{sub} \text{ 且 } v \in V_{sub}$
- 一个顶点覆盖的规模就是 V_{sub} 中包含的节点数，数量最少的集合称为最小顶点覆盖。

近似算法

```
void approxVertexCover(Graph g)
```

```
{
```

(1) 定点 C 集合初始化为空;

(2) 边集合 E' 初始化为图 G 的边集合

(3) 如果 E' 不为空

 从 E' 中选取一条边 $\langle u, v \rangle$

 将点 u 和 v 加入到集合 C 中 // 注意是将两个顶点都加入到集合 C

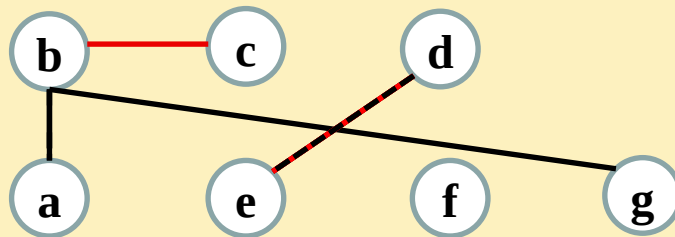
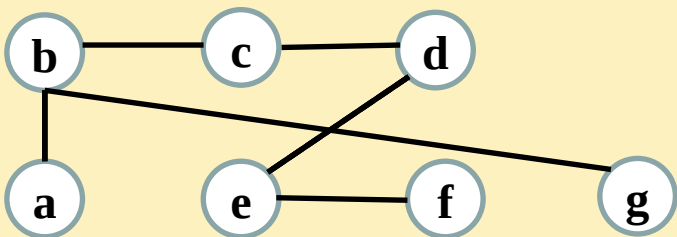
中

 从 E' 中删除所有以 u 和 v 为节点的边

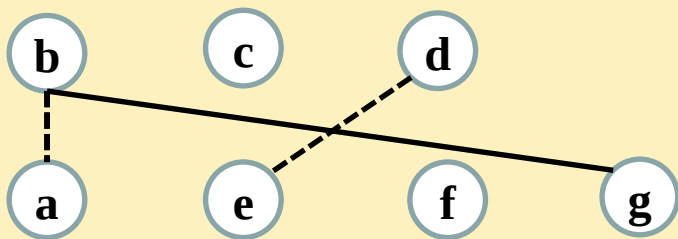
(4) 返回集合 C ;

```
}
```

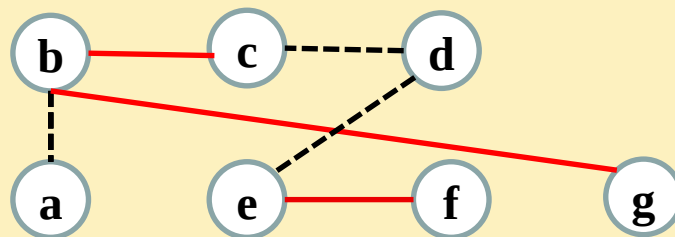
示例



$C = \{b, c\}$



$C = \{b, c, e, f\}$



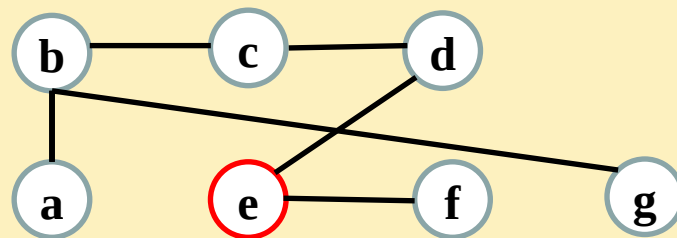
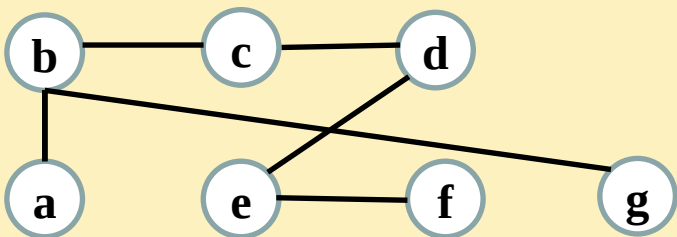
$C = \{b, c, e, f, d, g\}$

算法分析

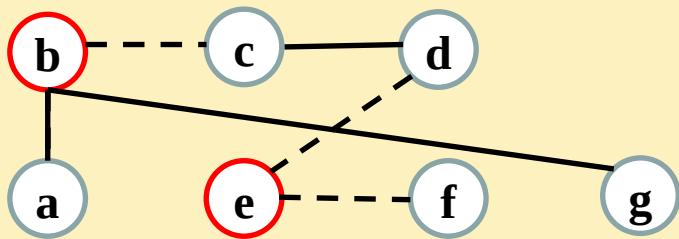
- 当确定选择一条边后，
 - 将这条边的两个顶点放到最小顶点集合中，
 - 删除与这两个顶点相关联的边
- 此时剩下的又是一个图，需要重复上面的步骤；
- 显然是一个递归的操作。



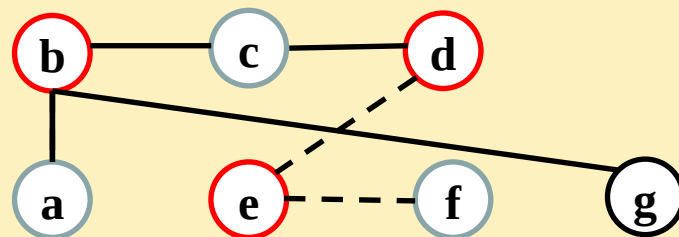
最佳的顶点覆盖——贪心



$C=\{e\}$ 节点 e 度最大



$C=\{e, b\}$



$C=\{e, b, d\}$

结果分析

- 最小顶点覆盖中顶点数是近似算法中顶点的一半
- 对于一个只有一条边的图，近似算法，包含两个节点，但是最小覆盖只需要任意一个顶点
- 猜想：存在一个近似比不超过 2 的解。



算法分析

- 顶点覆盖有一个多项式时间的 2 近似算法
- 设 A 为 **approxVertexCover** 中第 4 行选出的边集合。
- 最优覆盖 C^* ，都必须至少包含 A 中每条边的一个端点。如果一条边一旦在第 4 行中被选中，那么在第 6 行就会从 E' 中删除所有与其端点关联的边，因此， A 中不存在两条具有共同的端点边。 A 中不存在由 C^* 中的同一顶点所覆盖两条边，意味着对于 C^* 中每一个顶点， A 只有一条其关联边，由此得到最优顶点覆盖的规模下界。最好的情况就是一条边有一个节点在 C^* 中。
- $|C^*| \geq |A|$ 此时图 G 是一条边。或者是 n 条边各不相连的图。

算法分析

- 第 4 行的每一次执行都会挑选出一条边，其两个端点都不在 C 中，因此，所返回顶点覆盖的规模上界（实际上是一个精确的上界）为： $|C|=2|A|$
- 如果图此时包含 4 个节点，如果包含 6 条边，那么需要 3 个节点
- 因此顶点覆盖有一个多项式时间的 2 近似算法

近似算法的典型应用例——旅行商问题

- 旅行商问题的图改为带权无向连通图 $G=(V, E)$ ，其每一边 $(u, v) \in E$ 有一非负整数费用 $c(u, v)$ 。现在要找出 G 的最小费用哈密顿回路。



算法分析

- 旅行商问题中费用函数 c 具有三角不等式性质，即对任意的 3 个顶点 u 、 v 、 $w \in V$ ，有 $c(u, w) \leq c(u, v) + c(v, w)$ 。对于给定的带权无向图 G ，可以利用图 G 的最小生成树的来找近似最优的旅行商问题回路。

算法

```
void approxTSP(Graph g)
```

```
{
```

(1) 选择 g 的任一顶点 v ;

(2) 用 Prim 算法找出带权图 g 的一棵以 v 为根的最小生成树 $tree$;

(3) 采用深度优先遍历树 $tree$ 得到的顶点表 $path$;

(4) 将 v 加到表 $path$ 的末尾, 按表 $path$ 中顶点次序组成哈密顿回路

H ,

作为计算结果返回 ;

```
}
```

算法时间复杂度分析

- Prim()、DFS 算法的时间复杂度都是多项式级，所以该算法的时间复杂度也是多项式级。
- 从每个顶点出发这样求解，通过比较求出近似最优解，其时间复杂度仍然是多项式级的。

近似算法的典型应用例——0-1 背包

- 有 n 个重量分别为 $\{w_1, w_2, \dots, w_n\}$ 的物品，它们的价值分别为 $\{v_1, v_2, \dots, v_n\}$ ，给定一个容量为 W 的背包。
- 设计从这些物品中选取一部分物品放入该背包的方案，每个物品要么选中要么不选中，要求选中的物品不仅能够放到背包中，且物品重量和不超过 W 的最大价值的装入方案。

算法

```
void approx01Package(Graph g)
```

```
{
```

(1) 对所有物品计算其单价;

(2) 按照单价从高到底进行排序, 排序结果为 price 数组 ,price[i] 代表单价第 i 高的物品编号。

(3) 遍历 price 数组

 如果 $cur_w + w[price[i]] \leq W$ 将物品加入到背包中

(4) 返回集合 C;

```
}
```

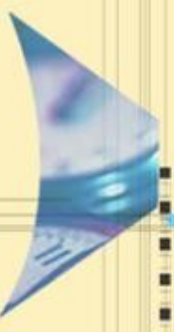
近似比等于 2

- 选择了前 k 个物品, $sol = \sum_{i=1}^k v_i$
- 对于第 $k+1$ 个物品, 只能装入部分, 假设 opt 是最优解

$$opt \leq \sum_{i=1}^k v_i + \frac{v_{k+1}}{w_{k+1}} (W - \sum_{i=1}^k w_i) < \sum_{i=1}^k v_i + \frac{v_{k+1}}{w_{k+1}} w_{k+1} = \sum_{i=1}^{k+1} v_i$$

$$sol = \max\{v_{k+1}, \sum_{i=1}^k v_i\} \geq \frac{1}{2} \sum_{i=1}^{k+1} v_i > \frac{1}{2} opt$$





算法分析

- 价值最大，那么希望单位价值（价值 / 重量）越大的物品优先装入背包。
- 对所有的物品按照单位价值进行排序



11.4 近似化策略总结

- 最先适配法 FF—— 序号最小的项优先选择。
- 最佳适配法 BF—— 选择一个策略，根据此策略进行排序，排在前面的项优先选择。
- 递减适配法 FFD—— 项首先按照序号的大小递减排序，然后用 FF 策略。
- 递减最佳适配法——

11.4 近似化策略总结

- 最先适配法 FF—— 序号最小的项优先选择。
- 最佳适配法 BF—— 选择一个策略，根据此策略进行排序，排在前面的项优先选择。
- 递减适配法 FFD—— 项首先按照序号的大小递减排序，然后用 FF 策略。
- 递减最佳适配法—— 选择一个策略，根据此策略进行排序，排在最后的项优先选择。